

AI IN ACTION-P1 · NGÀY 2

Xác Định Bài Toán Kinh Doanh Cho AI

Chọn đúng bài toán, đúng mức tự động hóa, đúng gate kỹ thuật

VinUniversity · Phase 1 · Tuần 1 · 2026



Nguyễn Tiến Đồng

Technical Director CMC AI

- 7+ năm nghiên cứu và phát triển các mô hình ML, AI
- 5+ năm ở vai trò quản lý quy trình nghiên cứu, phát triển các model ML, AI cho đến AI Product

Focus: AI product management, Natural language processing, Computer vision

Hôm nay bạn sẽ học gì?

3 câu hỏi xuyên suốt ngày:

- 1. Có thật sự nên dùng AI không?**
- 2. Nếu có, nên dùng mức nào: rule, feature, hay agent?**
- 3. Khi nào nên Go / Not Yet / No-Go?**

Sáng: học framework + case study → **Chiều:** tự làm với bài toán thật từ chính bạn

Flow buổi học

SÁNG — LECTURE (3H)

- Có nên dùng AI không? Buy vs Build
- Design Thinking + Human-Centered Design
- AI System + 5 Workflow Patterns
- AI Readiness Checklist + Problem Statement
- Go / Not Yet / No-Go

CHIỀU — LAB (3.5H)

-  **Cá nhân:** Scan 5+ bài toán từ chính mình
-  **Cá nhân:** Viết Quick Problem Card top 3
-  **Nhóm:** Pitch – Challenge – Vote → chọn 1
-  **Nhóm:** Deep-Dive: workflow, PS, research
-  **Nhóm:** Evaluate: AI Readiness → Go/No-Go
-  **Cá nhân:** Reflection Log

DELIVERABLES

➤ Deliverable 1

Scan & Filter Log (cá nhân)

➤ Deliverable 2

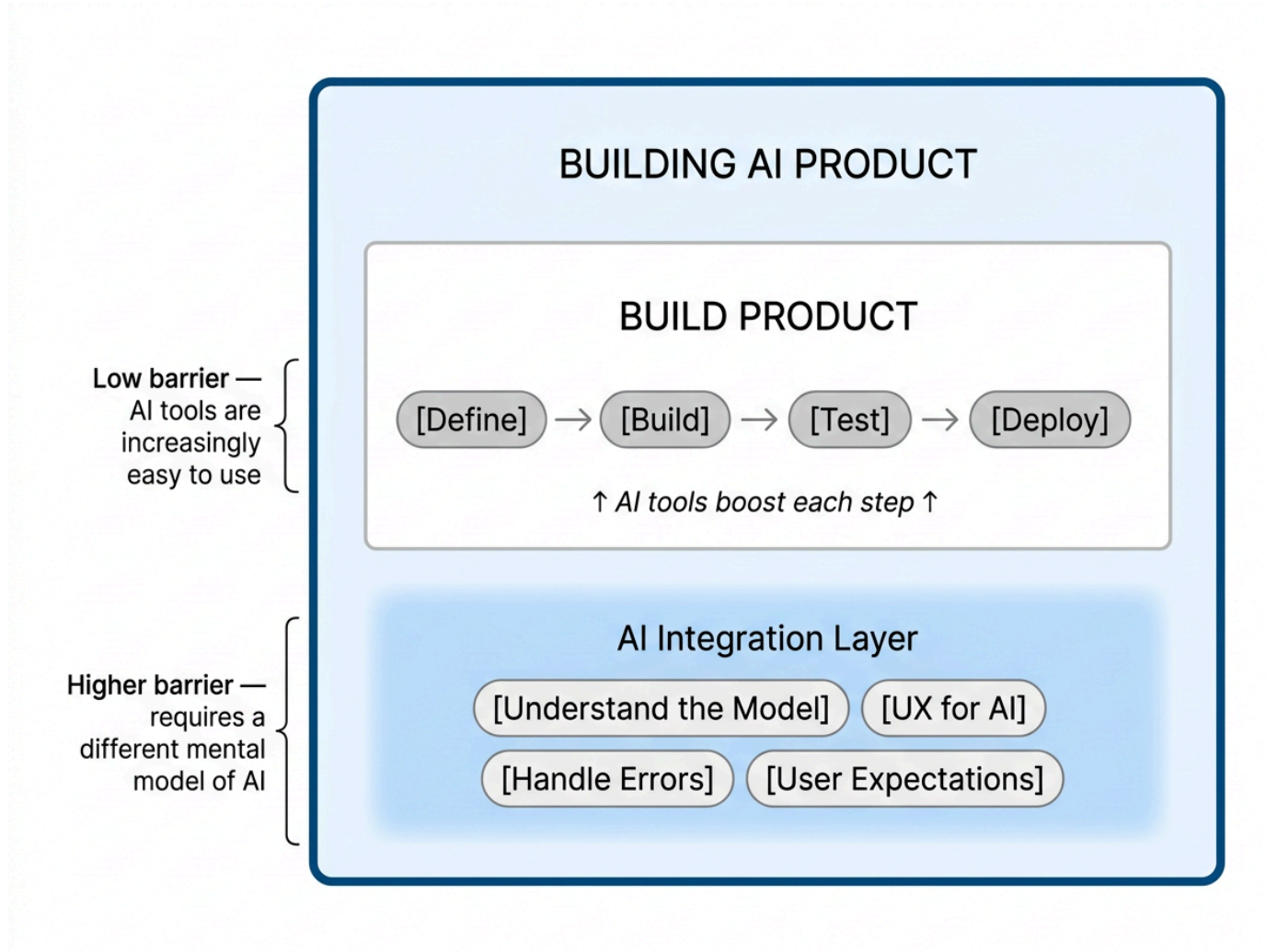
Problem Deep-Dive (nhóm)

➤ Deliverable 3

Reflection Log (cá nhân)

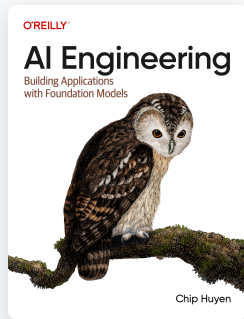
Building AI Product

AI Product chứa Build Product bên trong — không thay thế



Một Số Cuốn Sách Nền Tảng

Deep-dive thêm sau khóa học



KỸ THUẬT & HỆ THỐNG

AI Engineering

Chip Huyen (2025)

“Có API rồi — làm gì tiếp?” RAG, agents, guardrails, eval, production ops.

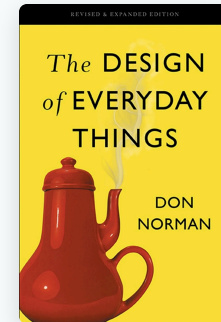


PRODUCT & STRATEGY

Inspired

Marty Cagan (2018)

Build cái gì, cho ai, và tại sao họ cần? Khám phá vấn đề thay vì feature factory.



TƯ DUY & THIẾT KẾ

Design of Everyday Things

Don Norman (2013)

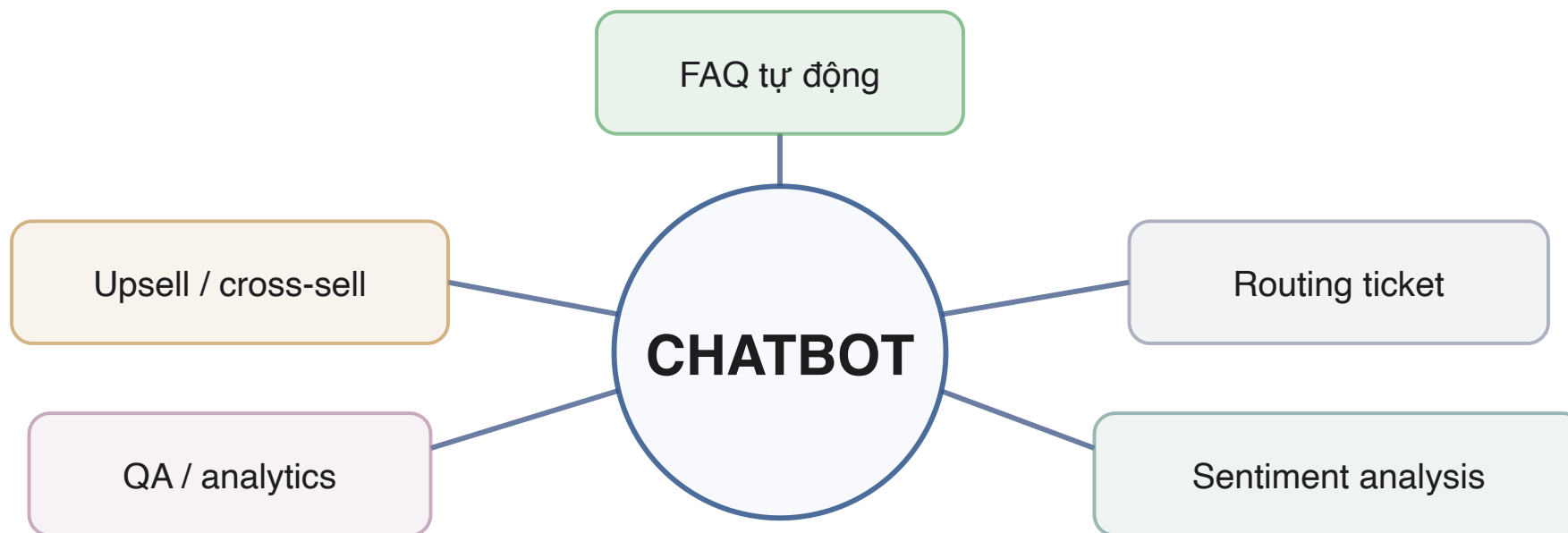
Affordance, feedback, mental model. Nền tảng UX — đặc biệt quan trọng khi AI không giải thích được.

“Tôi muốn xây chatbot cho khách hàng”

Pause & Think

Nếu stakeholder nói "Tôi muốn làm chatbot", theo bạn họ thực ra đang muốn giải bài toán gì?

“Chatbot” không phải 1 bài toán



1 brief = 5 bài toán = 5 kiến trúc = 5 metric khác nhau

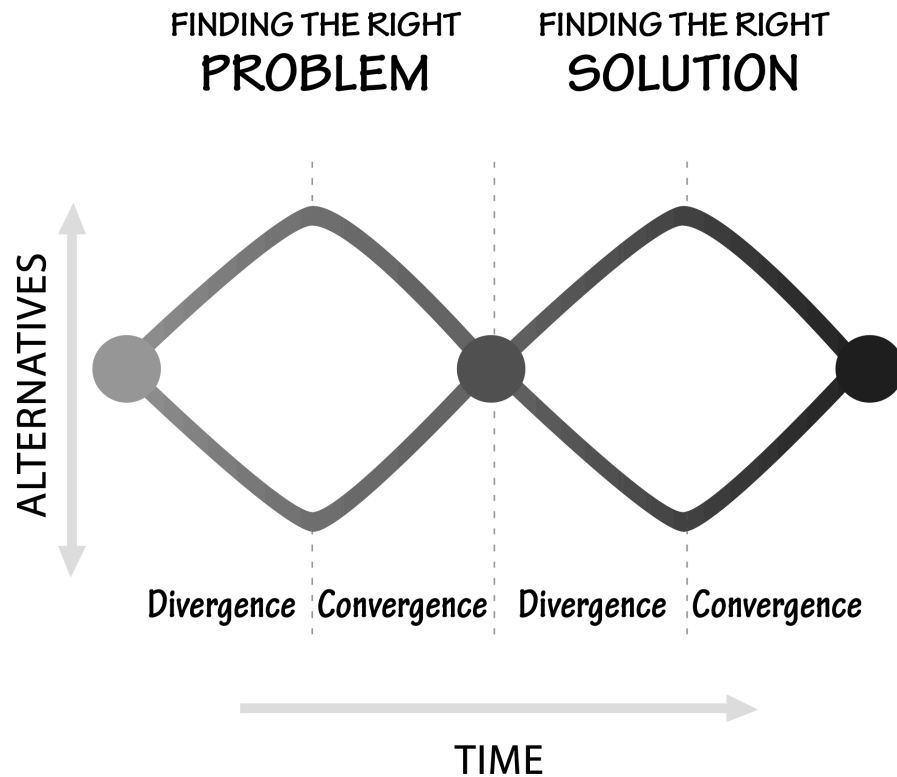
**AI không sai.
Họ giải sai bài toán.**

**“Đừng bao giờ giải quyết vấn đề
mà bạn được yêu cầu giải quyết.”**

Don Norman — *The Design of Everyday Things* (2013)

Tìm Đúng Vấn Đề Trước Khi Tìm Giải Pháp

Mô hình Kim Cương Đôi — Don Norman / British Design Council (2005)



Kim cương 1 — Tìm đúng vấn đề

Discover: Mở rộng — khảo sát vấn đề căn bản

Define: Thu hẹp — xác định đúng bài toán gốc

Kim cương 2 — Tìm đúng giải pháp

Develop: Mở rộng — nhiều giải pháp tiềm năng

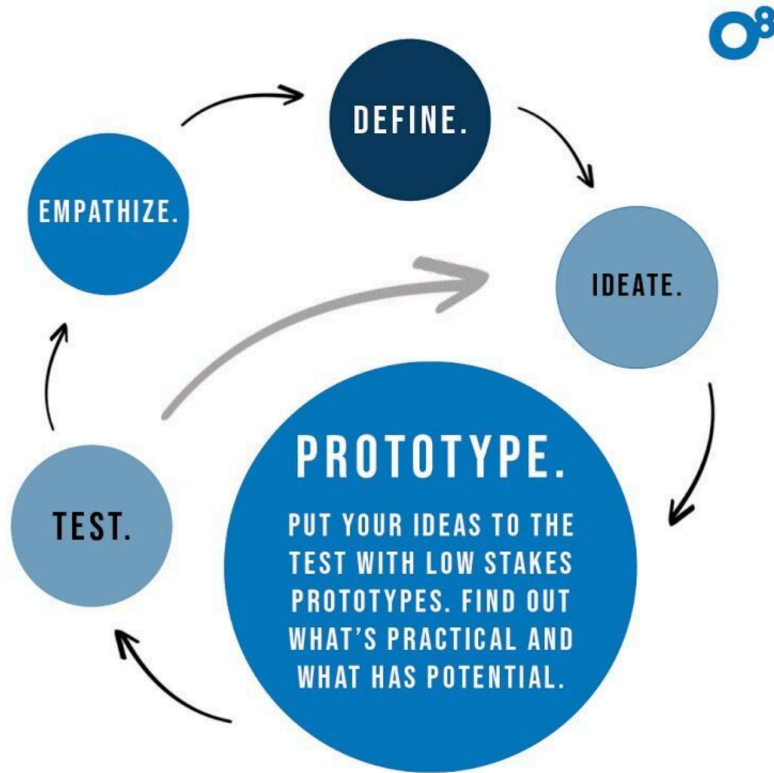
Deliver: Thu hẹp — chọn và triển khai

*Kỹ sư và doanh nhân được đào tạo để **giải** vấn đề.
Nhà thiết kế được đào tạo để **khám phá** vấn đề thật.*

Giải pháp xuất sắc cho sai vấn đề có thể còn tệ hơn không có giải pháp.

Quy Trình Thiết Kế Lấy Con Người Làm Trung Tâm (HCD)

4 bước lặp lại bên trong mỗi kim cương — Don Norman



Observation (Quan sát)

Những người được quan sát phải phù hợp với đối tượng mục tiêu.

Quan sát những khách hàng tiềm năng trong cuộc sống bình thường, hiểu các tình huống thực tế mà họ gặp phải.

Ideation (Tạo ra ý tưởng)

Tạo ra nhiều ý tưởng, sáng tạo mà không bị ràng buộc bởi các hạn chế.

Tránh phê bình ý tưởng của bản thân hay của người khác.

Đặt câu hỏi về tất cả mọi thứ.

Prototype (Tạo mẫu thử)

Tạo một nguyên mẫu nhanh cho mỗi giải pháp tiềm năng.

Mục tiêu là kiểm tra nhanh nhất có thể trước khi build.

Test (Kiểm tra)

Ngồi quan sát cách người dùng tương tác với Prototype trong thực tế.

Iteration (Lặp lại)

Tinh chỉnh và nâng cao liên tục.

Những Câu Hỏi “Ngớ Ngẩn” Đã Thay Đổi Thế Giới

Norman: “Hãy đặt câu hỏi về tất cả mọi thứ — đặc biệt là những điều hiển nhiên”



“Nếu quá táo rơi xuống, thì mặt trăng có rơi không?”

Isaac Newton

Câu hỏi ngớ ngẩn về quả táo → nền tảng toàn bộ cơ học cổ điển & cách mạng công nghiệp.



“Tại sao con phải đợi lâu mới xem được ảnh?”

Jennifer Land, 3 tuổi (1943)

Câu hỏi của đứa trẻ → cha cô bé phát minh máy ảnh chụp lấy ngay Polaroid.



“Nếu cho thuê phòng trống cho khách du lịch thì sao?”

Brian Chesky & Joe Gebbia (2007)

Hai designer vật lộn tiền thuê nhà → Airbnb, định giá hơn 100 tỷ USD.

Problem-first, not AI-first

3 case studies thực tế

Team Cursor

Xây AI cho ngành mình không hiểu sâu (4 tháng trong cơ khí) → thiếu dữ liệu, thiếu insight domain.

Kết quả: Hủy ý tưởng, pivot về phần mềm.

Artifact

AI cá nhân hóa tin tức tốt, nhưng market dynamics yếu (thiếu viral loop tự nhiên).

Kết quả: Đóng cửa sản phẩm.

CEO Lovable (Summer Labs)

Công nghệ tốt nhưng yêu cầu đối tác phong làm việc của user hiện có để tích hợp.

Kết quả: Khó được thị trường chấp nhận.

Bài học

Đừng xây AI rồi mong mọi người tự tìm cách dùng. Hãy bắt đầu từ trải nghiệm end-to-end và xác định đúng điểm AI giải quyết vấn đề người dùng thực sự quan tâm.

Câu hỏi không còn là **HOW to build** — mà là **WHAT TO BUILD**

Tìm bài toán AI ở đâu?

4 Lenses — Scan from self



Lặp lại

Việc gì tôi/team làm đi làm lại
mỗi ngày/tuần?



Tốn thời gian

Việc gì mất nhiều thời gian
hơn lẽ ra nên mất?



AI có thể tốt hơn

Sản phẩm nào tôi dùng mà AI
có thể cải thiện?



Pain từ người khác

Đồng nghiệp/bạn bè hay phàn
nàn gì?

Lab hôm nay bắt đầu bằng scan from self — dùng 4 lenses này để quét xung quanh mình trước khi chọn bài toán.

Hai bài học từ Google

GOOGLE FLU TRENDS

2008: Tương quan 97.5% với CDC

2011-2013: **SAI 100/108 tuần**, ước tính cao >50%, bỏ lỡ H1N1

Nguyên nhân: Proxy metric sai — search vì sợ cúm ≠ bị cúm thật

BÀI HỌC

Sai problem framing / sai proxy metric

Correlation đẹp lúc đầu ≠ đúng dài hạn

GOOGLE PHOTOS

Câu hỏi: Có nên dùng AI cho filter ảnh?

Đánh giá: Rule-based đã đủ tốt — nhanh, ít rủi ro, dễ kiểm soát

Quyết định: Chọn **KHÔNG** dùng AI

BÀI HỌC

Không phải chỗ nào AI cũng add value

Rule-based đủ tốt → đừng tăng complexity

Trước khi hỏi “dùng model gì” — phải hỏi “mình đang tối ưu đúng biến chưa” và “AI có thực sự tạo thêm value không”

4 Anti-Patterns Làm Team Đốt Tiền Vào AI Sai Chỗ

- ☐ **Trend-first:** Thay đổi theo trend “agent” trước khi rõ actor, workflow, metric
 - ☐ **No baseline:** Không có rule/manual baseline để so sánh nhưng vẫn build AI trước
 - ☐ **No eval path:** Có demo đẹp nhưng không có bộ test, không biết khi nào đủ tốt để deploy
 - ☐ **No owner of failure:** Không rõ ai review output sai, ai rollback, ai chịu compliance
- ☒ **Nguyên tắc:** Dừng AI khi nó tạo giá trị hơn cách đơn giản hơn, không phải vì nó nghe hiện đại hơn

Nhiều team nghĩ mình đang build AI. Thực ra họ đang build mơ hồ.

Discovery Interview: 5 Câu Hỏi Nên Hỏi Stakeholder

- **Pain point là gì?** Xảy ra bao nhiêu lần / ngày / tuần?
- **Workflow hiện tại ra sao?** Tool nào, bước nào, ai hand-off cho ai?
- **Cost của vấn đề là gì?** Mất bao nhiêu phút, tiền, SLA, hay conversion?
- **Nếu AI sai thì sao?** Điểm nào cần HITL, approval, hoặc chỉ để suggest?
- **Ai sẽ nói YES?** Metric nào và risk nào quyết định việc đầu tư?

Lưu ý: Nếu stakeholder không mô tả được workflow hiện tại và failure cost, team đang đề xuất solution trong sương mù.

Có nên dùng AI không và dùng mức nào?

Khi nào AI đáng để làm?

AI HỢP KHI NÀO

- Tác vụ lặp lại nhưng biến thể vừa phải
- Cần tổng hợp / search
- Nhiều bước + nhiều tool
- *Deterministic thì rule có thể tốt hơn*

VÌ SAO DOANH NGHIỆP ĐẦU TƯ

01 Sống còn — không dùng AI → bị đối thủ vượt

02 Hiệu quả — giảm chi phí, tăng tốc, cải thiện throughput

03 Khám phá — đầu tư để học, tránh tụt hậu, tìm cơ hội mới

Lý do áp dụng AI sẽ quyết định cách build, mức tự động hóa và mức đầu tư.

Tự xây dựng hay mua giải pháp?

Hai góc nhìn bổ sung nhau

GÓC NHÌN 1 — CHIP HUYEN, AI ENGINEERING (2025)

In-house (Build)

Khi AI là **lợi thế cốt lõi** và yếu tố **sống còn**

Mua / SaaS (Buy)

Khi AI chủ yếu là **productivity layer**

GÓC NHÌN 2 — MIT CISR (VAN DER MEULEN & WIXOM, 2025)

Buy

- Off-the-shelf, vendor duy trì
- Nhanh, ít differentiation
- Phụ thuộc vendor roadmap

Boost

- Mua model + enhance bằng data riêng
- Fine-tune hoặc RAG
- Cần data governance tốt

Build

- Tự xây custom model
- Control cao nhất, đắt nhất
- Cần team AI mạnh

Thực tế: Hầu hết team đang ở giữa — **Boost** (RAG / fine-tune). Không phải lúc nào cũng cần build from scratch.

Thiết lập kỳ vọng

Đo gì để biết AI có đáng triển khai?

Trước khi build, phải biết thế nào là “đủ tốt”

1 — TÁC ĐỘNG KINH DOANH

Question: AI tạo giá trị gì cho doanh nghiệp?

Đo bằng:

- ✓ % tác vụ / tin nhắn tự động hóa
- ✓ khả năng xử lý tăng thêm
- ✓ tốc độ phản hồi cải thiện
- ✓ thời gian lao động tiết kiệm

2 — SỰ HÀI LÒNG KHÁCH HÀNG

Question: Người dùng có thật sự thấy tốt hơn không?

Đo bằng:

- ✓ CSAT (Điểm hài lòng khách hàng)
- ✓ phản hồi trực tiếp
- ✓ hành vi sử dụng / bỏ giữa chừng / phải retry

3 — NGƯỠNG HỮU DỤNG

Question: Tới mức nào thì sản phẩm đủ tốt để ship?

Đo bằng:

Chất lượng: đầu ra có đúng và hữu ích không?

Độ trễ: TTFT, TPOT, thời gian phản hồi

Chi phí: mỗi request tốn bao nhiêu?

Từ Demo đến Production

Vì sao AI mất thời gian thật sự?

DEMO

1 cuối tuần

- ✓ Verify giả thuyết
- ✓ Align stakeholder
- ✓ Test nhanh UI/UX

PRODUCTION

Nhiều tháng → năm

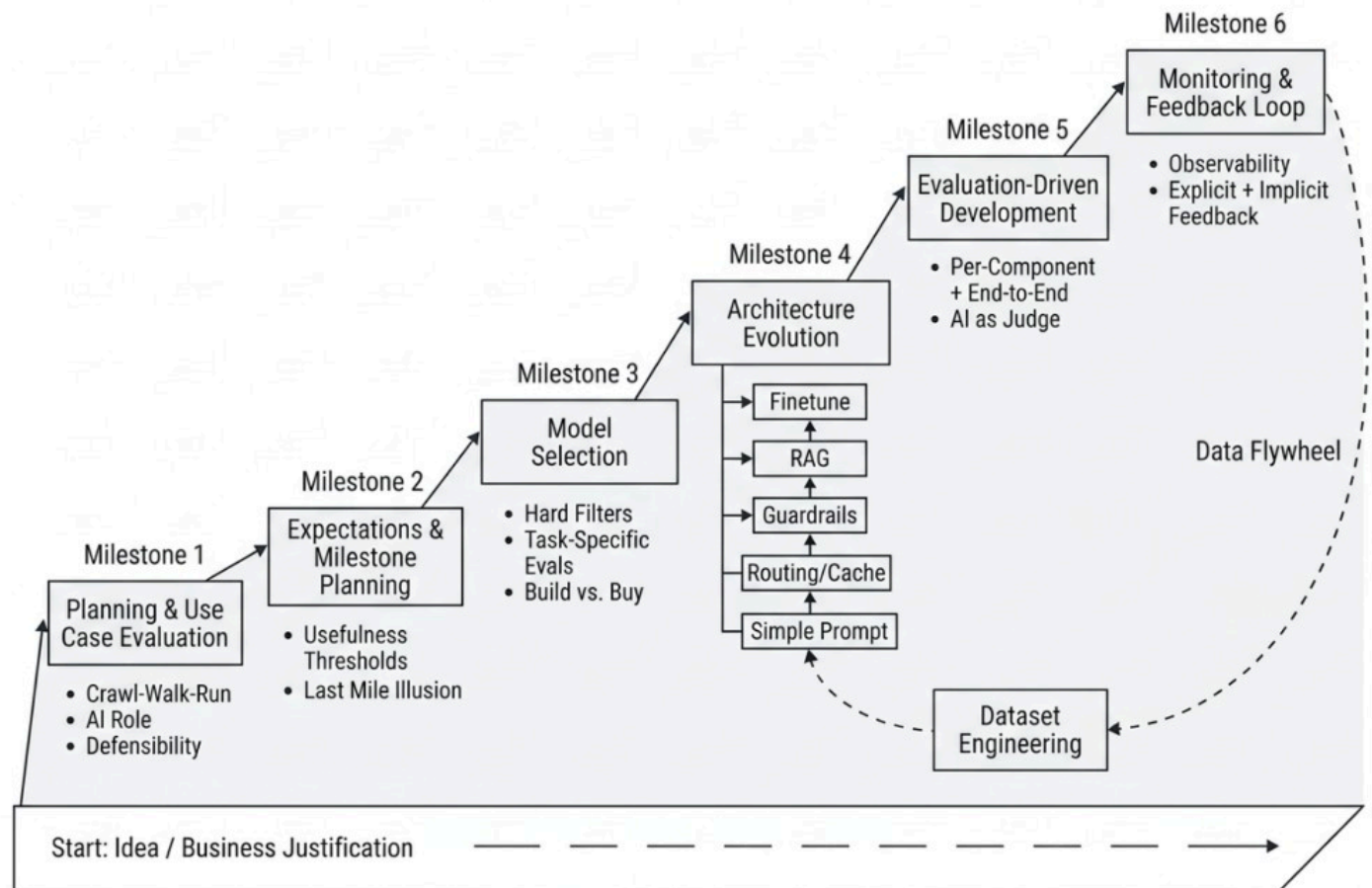
- ✗ Edge cases, guardrails
- ✗ Hallucination
- ✗ Giành từng 1% chất lượng

80% → 95% là đoạn đau nhất

LinkedIn: 80% trải nghiệm trong 1 tháng, mất thêm 4 tháng để nhích lên 95%

AI Product Lifecycle

6 milestones từ ý tưởng đến vận hành



Data Flywheel: feedback từ production → cải thiện data → model → architecture

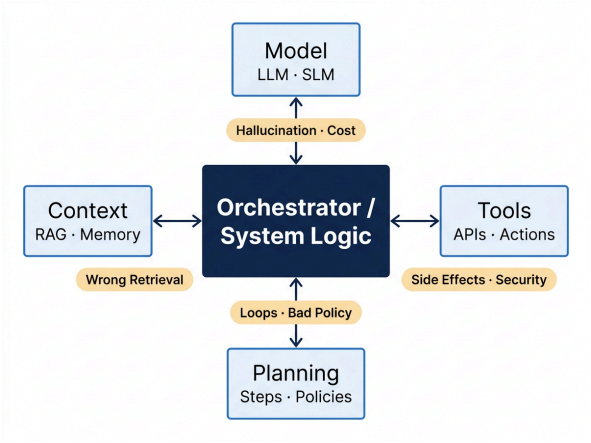
Vai Trò Của UX Trong AI Product

AI không cần hoàn hảo, nếu UX đỡ được chỗ nó yếu

 Không chắc (low confidence)	→	Xin user xác nhận trước khi thực hiện
 Risk cao (sai = hậu quả nghiêm trọng)	→	Chỉ suggest, không auto-action
 Câu trả lời dài (quá tải thông tin)	→	Chia option / card / summary cho user chọn
 Thiếu context (input mơ hồ)	→	Hỏi lại đúng chỗ thay vì đoán sai

AI Product = AI + UX. Dùng UX để hỗ trợ chỗ AI chưa đủ tốt.

AI System = Model + Context + Planning + Tools



Thành phần	Cần khi...	Failure mode	Control	Cost driver
Model	Task mở, cần suy luận / tạo văn bản	Hallucination, inconsistency	Eval set, guardrails, structured output	tokens / latency
Context	Cần dựa trên tài liệu / trạng thái ngoài	Wrong retrieval, stale context	Retrieval test, freshness, citations	storage / retrieval
Planning	Task nhiều bước, cần ra quyết định trung gian	Looping, over-planning	Step limit, policy, approvals	extra calls
Tools	Cần đọc / ghi hệ thống ngoài	Side effects, prompt injection	sandbox, allowlist, HITL	API calls / failures

Framework Chọn Rule / Workflow / Agent

Bắt đầu đơn giản nhất — chỉ tăng phức tạp khi bài toán thật sự cần

04

Ba Mức Giải Pháp: Rule / Workflow / Agent

Rule / Script

- Input ổn định
- Logic rõ ràng
- Cần predictability cao
- Compliance nặng

LLM Feature

- Input biến thể vừa phải
- Output cần linh hoạt
- Có metric và guardrails
- Human có thể review

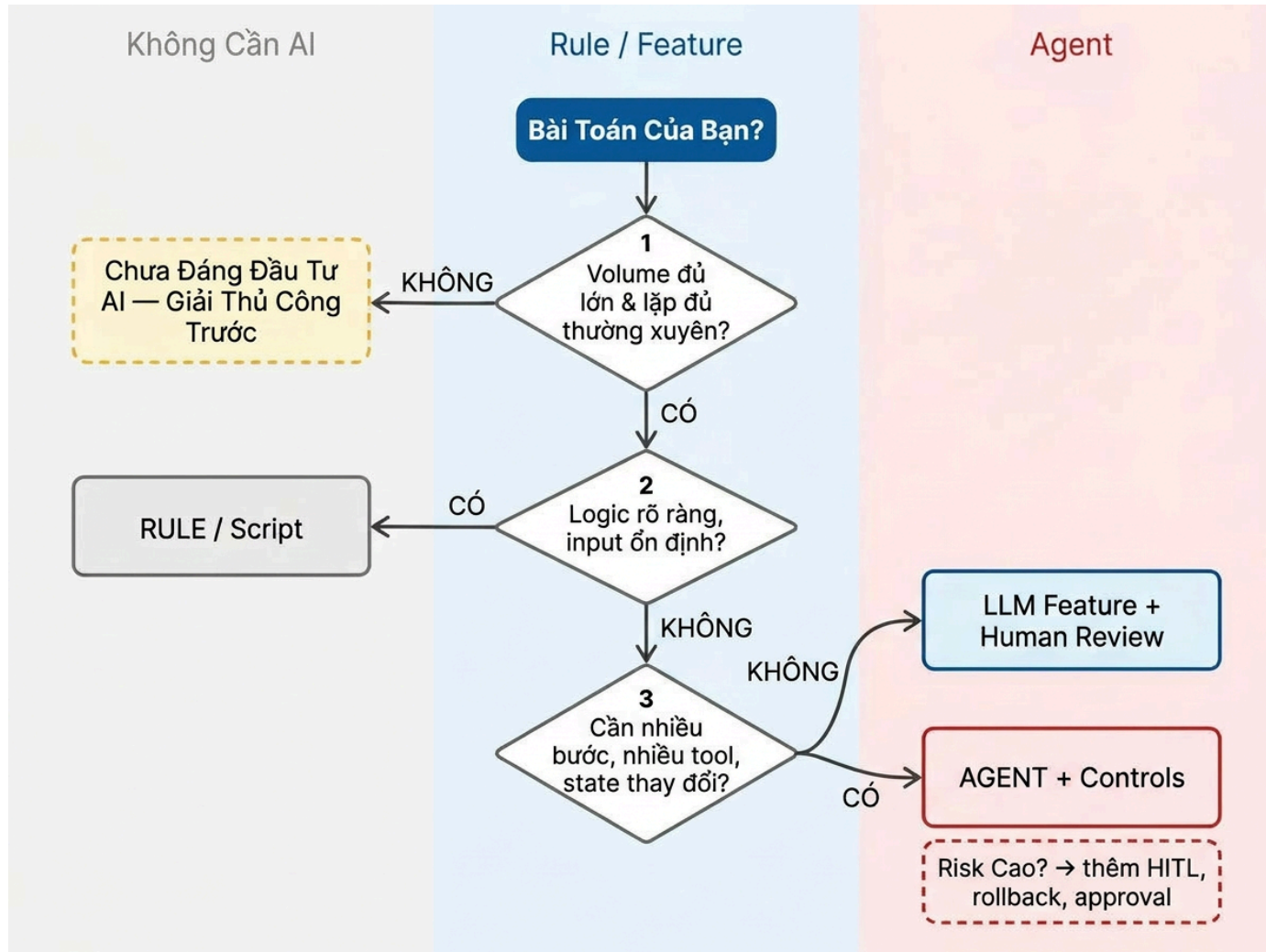
Agent

- Nhiều bước, nhiều tool
- Trạng thái thay đổi liên tục
- Cần quyết định động
- Có risk control rõ ràng

Thứ tự ưu tiên thực dụng: bắt đầu từ bên trái, chỉ đi sang bên phải khi giá trị tăng hơn độ phức tạp.

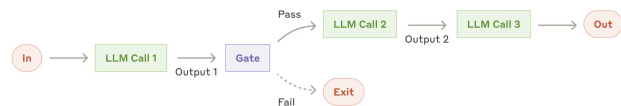
Decision Tree: Chọn mức giải pháp

Từ bài toán của bạn → Rule, LLM Feature, hay Agent?



Workflow Patterns — Đủ cho hầu hết bài toán

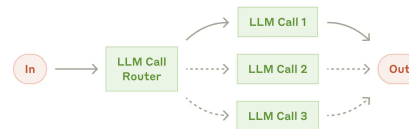
Nguồn: Anthropic — Building Effective Agents (2024)



1. Prompt Chaining

Chia task thành chuỗi bước tuần tự. Có **gate** kiểm tra giữa các bước.

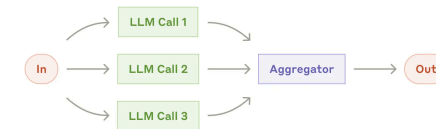
VD: Viết outline → check → viết bài



2. Routing

Phân loại input → đưa vào nhánh chuyên biệt. Tối ưu từng loại riêng.

VD: CS query → FAQ / refund / kỹ thuật



3. Parallelization

Chạy song song rồi tổng hợp (**sectioning**), hoặc chạy nhiều lần lấy **vote**.

VD: Guardrail + response đồng thời

Nguyên tắc Anthropic: “Start with the simplest solution possible, only increase complexity when needed.” — Hầu hết bài toán thực tế chỉ cần 3 patterns này.

Khi Nào Cần Phức Tạp Hơn?

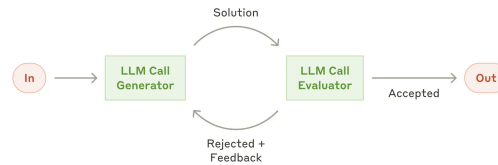
Orchestrator-Workers, Evaluator-Optimizer, và Agent



4. Orchestrator-Workers

1 LLM phân việc động cho workers. **Subtasks không biết trước.**

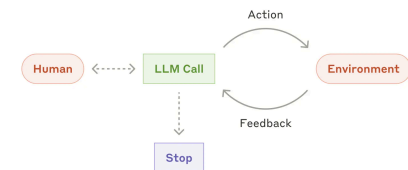
VD: Coding agent sửa nhiều file



5. Evaluator-Optimizer

1 LLM tạo, 1 LLM đánh giá → **lặp cho đến khi đạt.**

VD: Dịch văn học → review → sửa



Agent

LLM tự lập kế hoạch + gọi tools + iterate. **Autonomous loop.**

VD: SWE-bench, computer use

Chi phí cao hơn, lỗi cộng dồn. Chỉ dùng khi không predict được subtasks và cần tool autonomy. “Agents’ autonomy makes them ideal for scaling tasks in trusted environments.”

AI Readiness Checklist - 5 Câu Hỏi Nhanh

- ✓ **Value:** Bài toán xảy ra thường xuyên và đang tốn thời gian / tiền / SLA thật sự?
- ✓ **Baseline:** Hiện tại đã có manual hoặc workflow baseline để so sánh?
- ✓ **Eval:** Có metric, sample cases, hoặc logs để đánh giá reproducible?
- ✓ **Tolerance:** Sai số hữu hạn có chấp nhận được, hoặc có HITL ở điểm quan trọng?
- ✓ **Operations:** Có owner, rollback, logging, và policy cho output sai?

✗ Dưới 3 câu YES: dừng lại và làm rõ problem / workflow trước khi đầu tư AI

Thiếu baseline hoặc thiếu eval = chưa biết AI đang tốt hơn cái gì

Lưu ý: AI "Jagged Frontier" — ranh giới AI gồ ghề, khó đoán. Framework là starting point, không phải kết luận. Phải test thực tế. (Dell'Acqua, Mollick et al. 2023)

Gate Criteria: Đi Tiếp Khi Nào, Dừng Lại Khi Nào?

Rules of ML + practical AI product heuristics

Giai đoạn	Go nếu...	No-Go nếu...
Problem Scoping	Actor, workflow, pain, metric đã rõ	Problem mơ hồ, mô tả solution trước problem
Data Readiness	Có data / logs / docs / SME support để eval	Không có nguồn dữ liệu hoặc dữ liệu quá lệch
Baseline / Model	Heuristic, workflow, hoặc baseline người được xác lập	Chưa biết AI cần tốt hơn cái gì
Build & Eval	Có bộ test, task-level eval, latency/cost budget	Chỉ có demo thủ công, không có reproducible eval
Deploy Controls	Có HITL, logging, approval, rollback, owner of failure	Không rõ ai duyệt output, ai dừng hệ thống khi sai

Key takeaways:

- Không có metric → không có gate
- Không có baseline → chưa biết tốt hơn gì
- Không có eval → mới chỉ là demo
- Không có rollback → deploy là đánh bạc

Problem Statement

Từ ý tưởng mơ hồ đến bài toán có metric, boundary, và đường eval

05

Khung Problem Statement Cho AI System

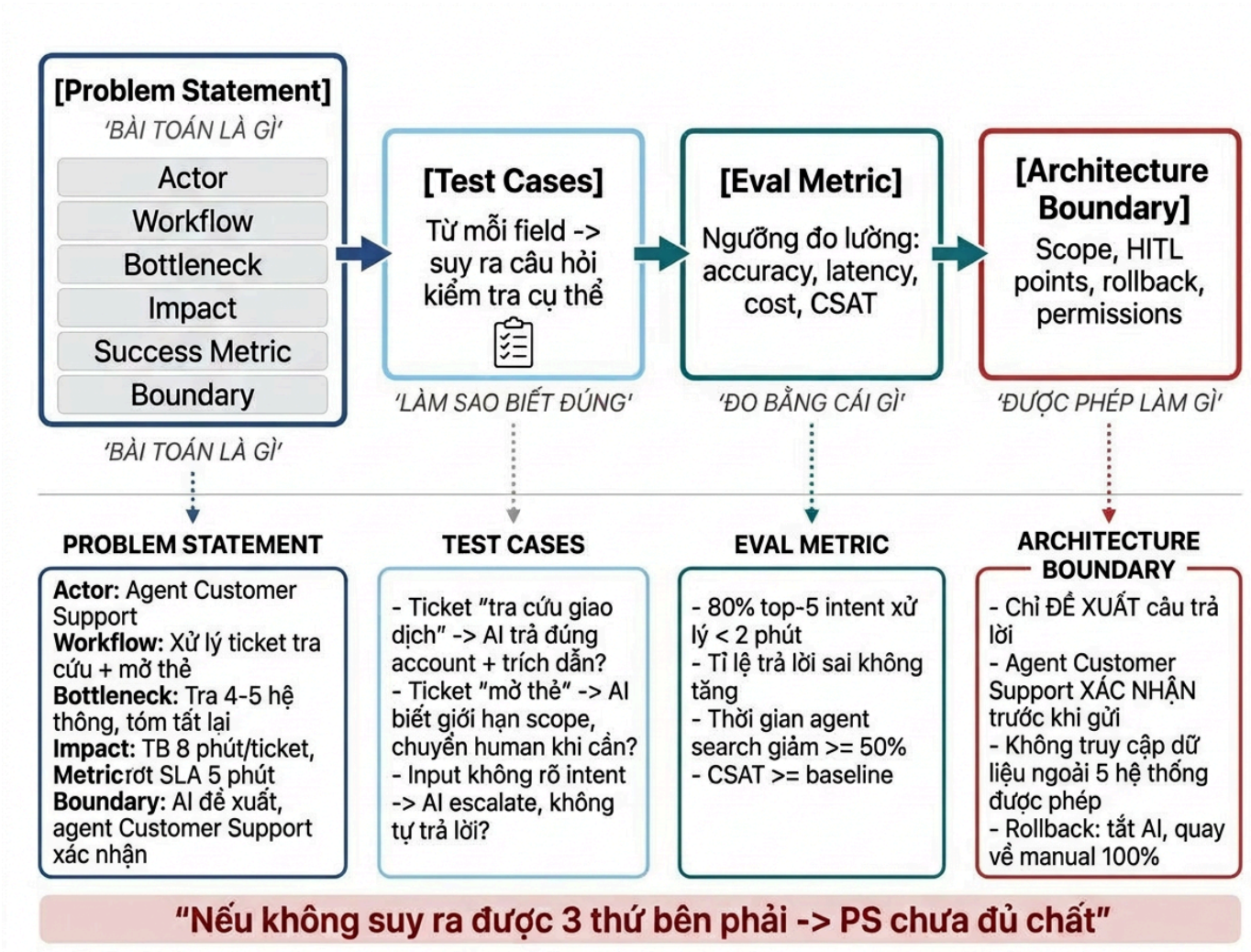
Một problem statement tốt phải giúp suy ra được test case, metric và boundary

Actor / Operator	Ai đang làm việc này hằng ngày?
Current Workflow	Hiện tại họ xử lý qua những bước nào, dùng tool gì?
Bottleneck	Bước nào chậm, lỗi, không nhất quán, hoặc cần tổng hợp quá nhiều?
Impact	Tổn thất đo bằng thời gian, chi phí, SLA, error rate, hay conversion nào?
Success Metric	Khi nào được coi là thành công? Mức ngưỡng là bao nhiêu?
Operational Boundary	Hệ thống được phép làm gì, không được phép làm gì, và điểm nào cần HITL?

Nhớ

Nếu viết xong mà bạn chưa suy ra được test cases, eval metric, và architecture boundary, thì problem statement vẫn chưa đủ chặt.

Từ Problem Statement Sang Eval Plan

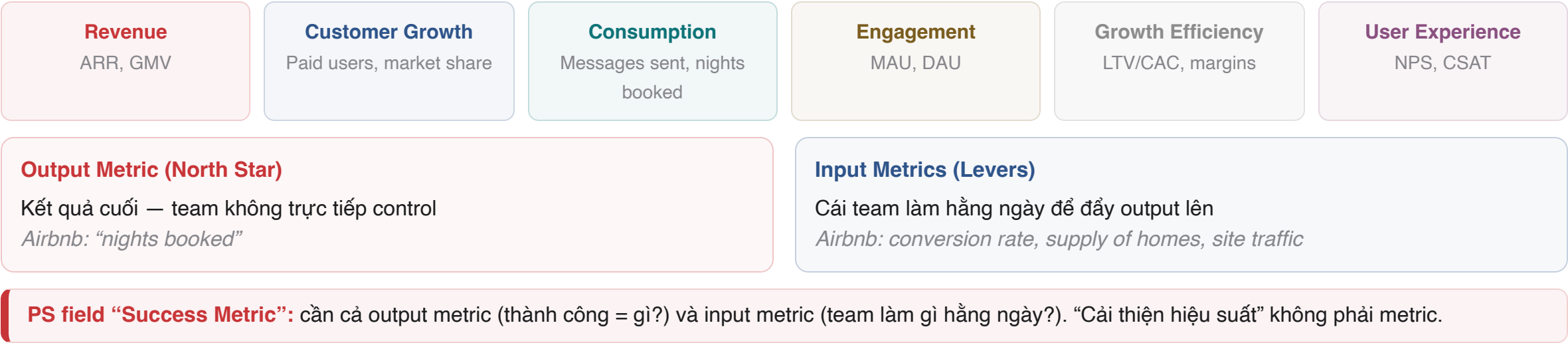


Problem Statement tốt là cầu nối giữa bài toán kinh doanh và bộ eval kỹ thuật.

Chọn Đúng Metric: North Star & Input/Output

Nguồn: Lenny Rachitsky (a16z) — North Star Metric Framework

Câu hỏi: metric nào nếu tăng hôm nay sẽ tăng tốc flywheel kinh doanh nhất?



Discovery và Feasibility

Kiểm tra bài toán, khả năng triển khai, và risk trước khi viết dòng code đầu tiên

07

Go / No-Go / Not Yet

- **Go:** Problem rõ, baseline rõ, eval rõ, risk controls rõ.
- **Not Yet:** Có pain thật, nhưng thiếu data, thiếu metric, hoặc chưa rõ workflow boundary.
- **No-Go:** Rule-based đã đủ tốt, hậu quả khi sai quá đắt, hoặc change cost lớn hơn value.

Bài học cho Dev

Quyết định chuẩn không phải lúc nào cũng là “build”. Rất nhiều dự án AI thành công bắt đầu bằng **không build ngay**, mà bằng đo baseline, gom logs, và thử workflow đơn giản hơn.

Lab hôm nay không bắt buộc ra kết luận Go. Kết luận tốt có thể là Not Yet hoặc No-Go.

Pause & Think

Workflow nào quanh bạn đủ đau để đáng cân nhắc AI?

Tổng Kết Ngày 2

- 1 Đừng giải quyết vấn đề bạn được yêu cầu giải quyết — hãy tìm vấn đề thật trước. (Don Norman)
- 2 Bắt đầu từ rule / workflow trước khi nhảy lên agent. Đúng architecture quan trọng hơn đúng model.
- 3 5 Workflow Patterns (Anthropic) cho vocabulary cụ thể — hầu hết bài toán chỉ cần Prompt Chaining hoặc Routing.
- 4 Problem Statement tốt phải suy ra được eval plan, system boundary, và success metric có ngưỡng cụ thể.
- 5 "Not Yet" không phải thất bại — đó là quyết định trưởng thành nhất khi chưa đủ data hoặc baseline.

Thực Hành

Lab 2: Chọn use case, viết PS, và ra quyết định go/no-go

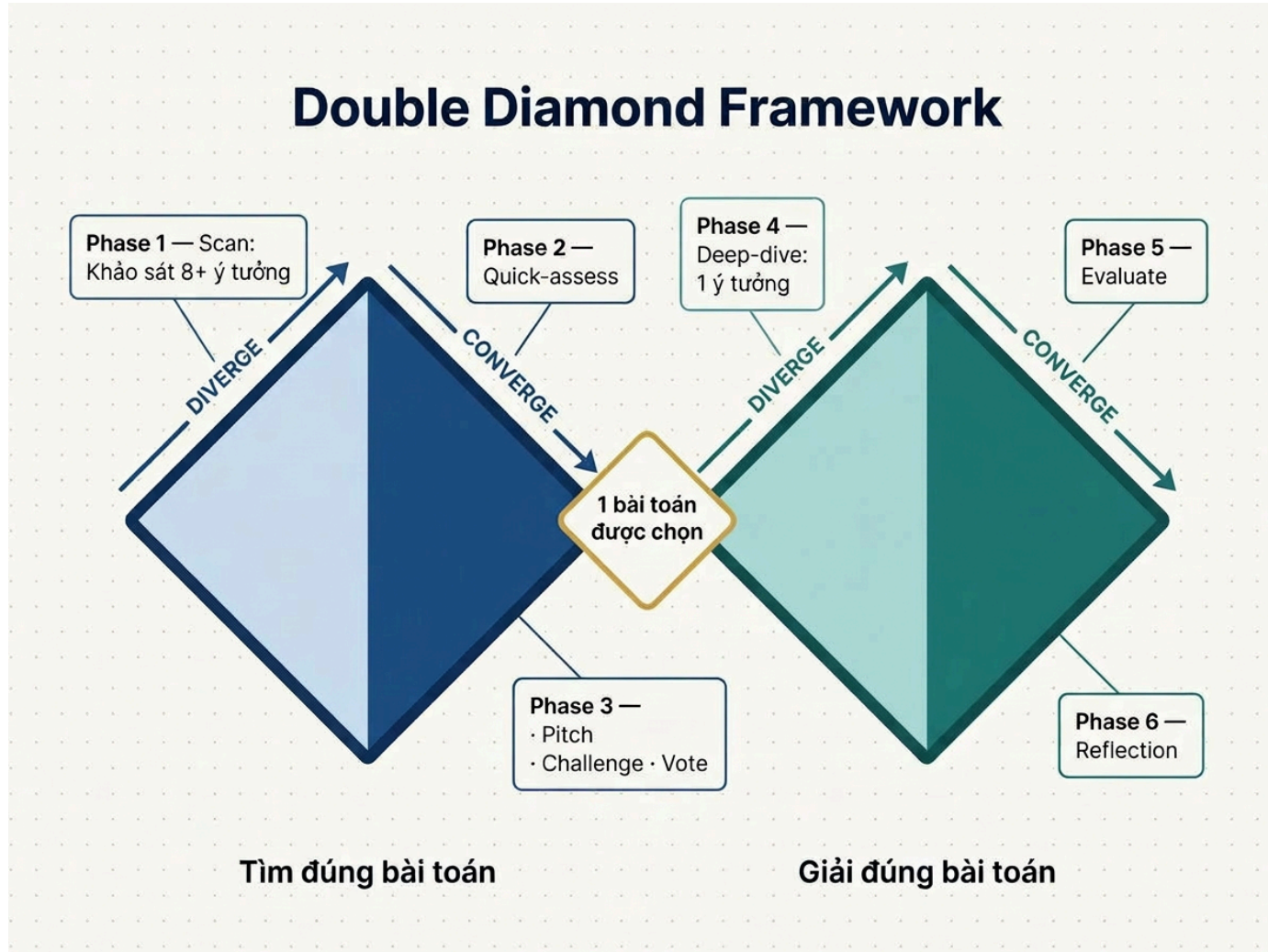
08

Tổng quan Lab 2 — Deliverables & AI Rules

PHASE	Phase 0 15 min	Phase 1 20 min	Phase 2 40 min	Break 10 min	Phase 3 30 min	Phase 4 85 min	Break 10 min	Phase 5 20 min	Phase 6 10 min
DELIVERABLE	 Hiểu flow mẫu	 8+ Ideas	 3 Quick Problem Cards		 1 bài toán được chọn	 Workflow + PS + Research + Future Flow		 Go / Not Yet / No-Go	 AI Support Log

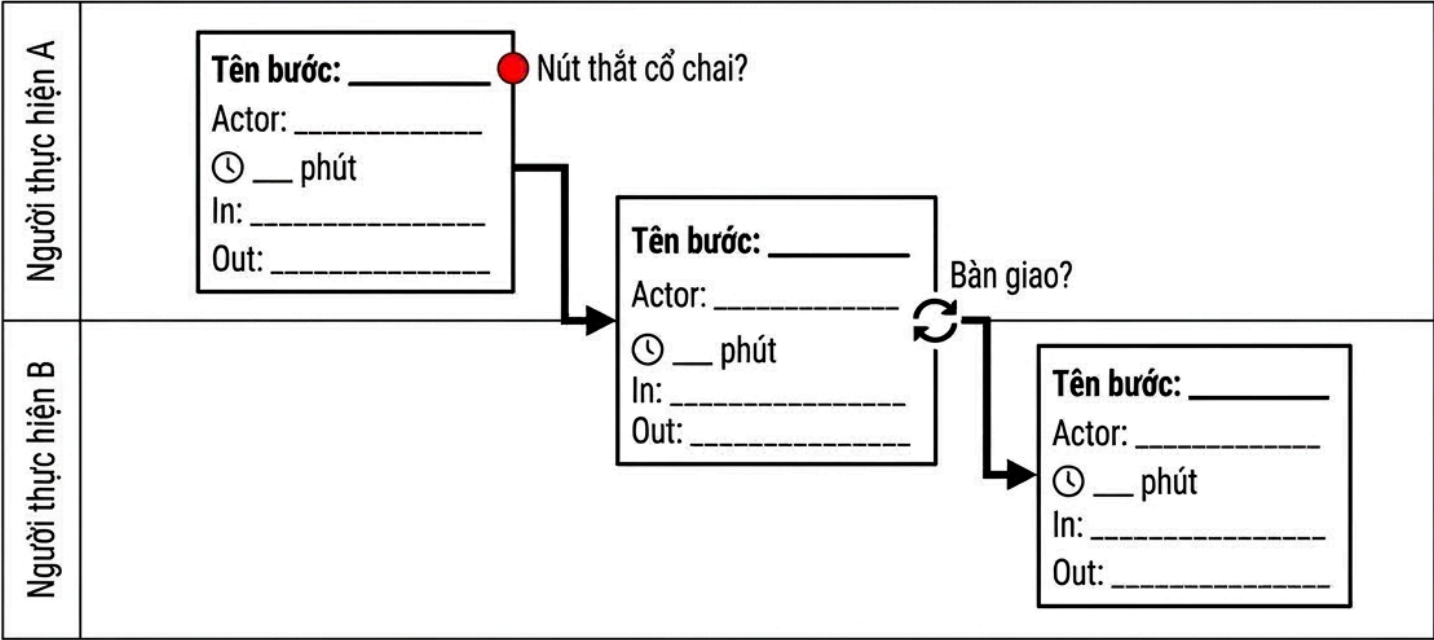
Phase 1 + 2

Hoạt động cá nhân



Hướng dẫn vẽ Workflow Diagram

Dùng cho Phase 4 — Deep-dive: Current-State & Future-State



KÝ HIỆU:

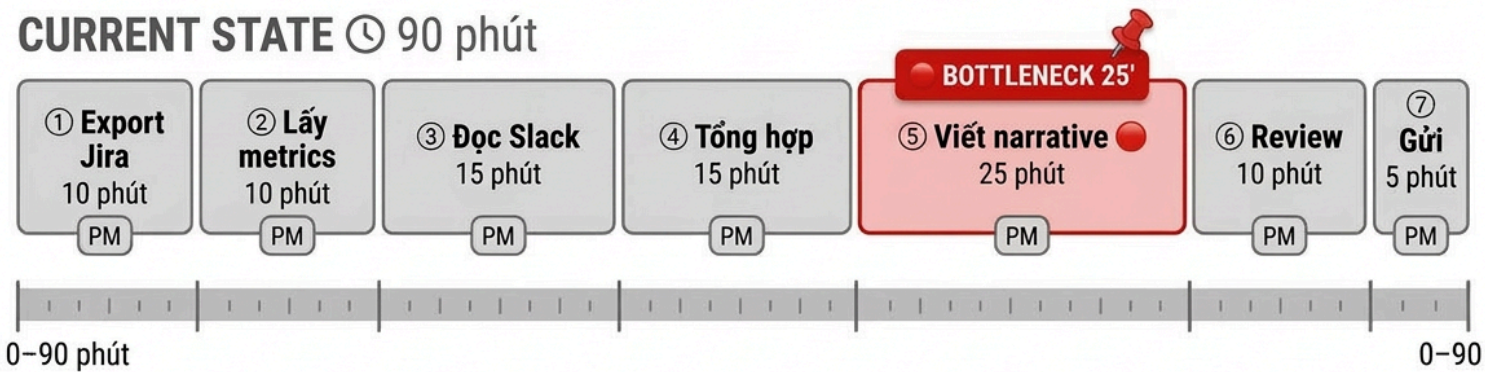
- = Nút thắt (bước chậm nhất hoặc hay xảy ra lỗi)
- ↻ = Bàn giao (điểm chuyển giao giữa người / hệ thống)
- 🕒 = Thời gian (ghi cụ thể, ví dụ: 15 phút)

- ☒ Tối thiểu 5-8 bước (đừng gộp quá)
- ☒ Ít nhất 1 nút thắt được đánh dấu
- ☒ Ghi tổng thời gian của workflow
- ☒ Each step has a specific time duration
- ☒ Mỗi bước có thời gian cụ thể
- ☒ Vẽ trên giấy, không gõ text

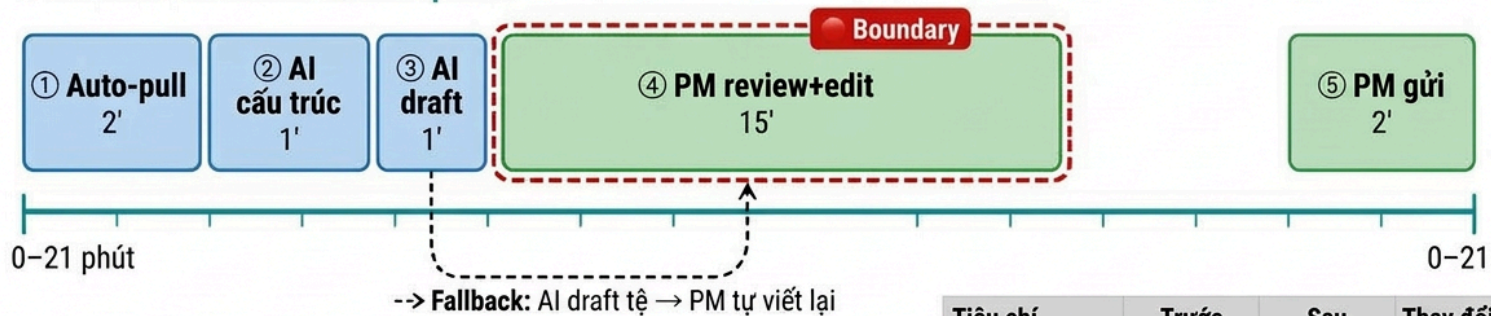
Worked Example: Weekly Report — Trước và Sau AI

Weekly Report - 7 bước, 90 phút → 5 bước, 21 phút

CURRENT STATE ⌚ 90 phút



FUTURE STATE ⌚ 21 phút



Tiêu chí	Trước	Sau	Thay đổi
Tổng thời gian	90 phút	21 phút	-77% ▼
Bước thủ công	7/7 thủ công	2/5 thủ công	—
Bottleneck mới	—	Review	✓

Phase 3 · Pitch – Challenge – Vote

Framework peer review trong nhóm · 30 phút

PITCH

2 phút / người

- Present bài toán của mình cho nhóm
- Dùng Quick Problem Card làm visual

CHALLENGE

3 câu hỏi chuẩn · 3 phút / người

- "Rule/script đủ chưa? Có thật sự cần AI không?"
- "Ngoài bạn, ai đau nữa? Bao nhiêu người?"
- "Metric đo được không? Có số cụ thể chưa?"

VOTE

5–10 phút sau khi tất cả pitch

- Mỗi người 1 phiếu → chọn 1 bài đi tiếp
- Viết 1 dòng kill rationale cho bài bị loại

Quick Problem Card — Ví dụ đã điền

Mỗi người điền 1 card trong Phase 2 · ~5 phút

Bài toán (1 câu)	Mất 3h/ngày để tổng hợp feedback từ 200+ email khách hàng thành báo cáo cho team sản phẩm
Ai đang đau?	Product Manager · ~50 PM trong công ty · lặp lại mỗi tuần
Workflow hiện tại (3-5 bước)	Nhận email → đọc từng email → tự tóm tắt → gộp thành doc → gửi team
Bước nào tốn nhất?	Bước 2 — đọc & tóm tắt (~2.5h/ngày · 150h/tuần × 50 PM)
AI có thể giúp ở bước nào?	Bước 2 — AI tóm tắt từng email, PM review + edit trước khi gửi
Đo thành công bằng gì? (có số!)	Giảm từ 3h → dưới 30 min/ngày · chất lượng báo cáo đồng đều hơn
Quick gut	☒ LLM — cần xử lý ngôn ngữ tự nhiên đa dạng, không thể rule-based



Nhận xét

Lab 2 — Deliverables — Mỗi người cần nộp gì?

[Deliverables Example](#)

LAB 2 · 4 giờ

Scan bài toán, filter, deep-dive, và quyết định go/no-go

1

Scan & Filter Log

Liệt kê 5+ bài toán (4 Lenses) → Quick Problem Card top 3 → Peer validate → Chọn 1

 Cá nhân

2

Problem Deep-Dive

Current-state flow + PS + metrics + research + future-state flow + AI fit + Go/Not Yet/No-Go

 Nhóm

3

Reflection Log

Ghi lại cách dùng AI trong quá trình lab: prompt, output, và chỗ phải sửa

 Cá nhân